# Network Security: Authentication, TLS and Firewalls

Lecture 31

Computer Networks and Distributed Programming

# From keys to protocols

- We have the pieces: symmetric and public-key ciphers, hashes, MACs, certification authorities. Now they must work together.
- Of the four properties of a secure communication, three are built into protocols: integrity, end-point authentication, confidentiality.
- Two pieces are still missing: a signature anyone can check, and the reason a certificate cannot be forged.
- Then: authentication protocols, and TLS, which puts all of it under a socket.
- Finally: the fourth property, at the edge of the network: firewalls and intrusion detection systems.

# Digital signatures

- A handwritten signature is verifiable and non-forgeable: it shows who signed, and nobody else could have.
- A MAC cannot do this: to check $H(m + s)$ Alice needs $s$, and with $s$ she could have computed it herself.
- Bob's private key is his alone. He signs with it: the signature is $K_B^-(m)$.
- Anyone checks it with his public key, $K_B^+(K_B^-(m)) = m$: the reverse property of RSA, seen with public keys.
- Change a single bit of $m$ and the check fails: a signature brings integrity with it.
- Public-key operations are slow, so Bob signs the hash: he sends $m$ together with $K_B^-(H(m))$.

# Signature or MAC

Alice computes $H(m)$ herself and compares it with $K_B^+(K_B^-(H(m)))$: equal means intact, and signed by Bob.

| | **MAC** | **Digital signature** |
|---|---|---|
| The key | a secret $s$ shared by both | Bob's private key, his alone |
| Who can check it | whoever holds $s$ | anyone with $K_B^+$ |
| Who could make it | either of the two | only Bob |
| Cost | one hash | a hash and a public-key operation |
| What it needs | a way to share $s$ | a certificate for $K_B^+$ |
| Used by | TLS records, OSPF | certificates, DNSSEC, PGP |

- A MAC convinces Bob. A signature convinces a third party: Alice can take it to a judge.

# The pizza that Bob never ordered

**Trudy orders a pizza in Bob's name, signs the order with her own private key and attaches her public key, saying it is Bob's. Does the signature check?**

Perfectly. Alice verifies with the key she was given, and a pizza with anchovies is delivered to Bob's door.

- A signature proves only that the message comes from whoever holds the matching private key.
- It is the question we left open with certification authorities: whose key is this?
- Something must bind a public key to a name, in a form Trudy cannot forge.

# Certificates, and who signs the signer

- The second job of a CA, issuing a certificate, is a signature: the pair $(K_B^+, \text{Bob})$, signed with $K_{CA}^-$.
- Alice checks it with $K_{CA}^+$. Change the key or the name and it fails; only the CA could have made it.
- X.509: issuer, subject, the subject's public key, validity period, serial number, signature algorithm.
- And who vouches for $K_{CA}^+$? A higher CA, with a certificate of its own: a chain, up to a root.
- Root keys come preinstalled with the operating system or the browser. The server sends the rest of the chain.
- It is still trust: a chain is as sound as the least careful CA your machine accepts.

# End-point authentication

## End-point authentication

One entity proving its identity to another over a network — to a party that is live, at the other end, now.

- A signature proves who wrote a message at some point; this proves who is there now. Only messages count: no face, no voice.
- An authentication protocol runs first; transfer, routing or mail start only after it.
- A certificate is not enough: it is public, and holding a copy proves nothing.
- Authentication: who you are. Authorization: what you may do — a table lookup, once the first question is settled.
- We build it as we built reliable transfer: one version at a time, and a hole in each.

# Trusting the source address

- "I am Alice" proves nothing: Trudy can send exactly the same message.
- If Alice always writes from the same IP address, Bob could check the source of the datagram.
- It stops only naive intruders. Writing any source address into a datagram is easy, as we saw with spoofing.
- The first-hop router could drop datagrams whose source is not one of its own hosts (RFC 2827).
- Not every network does it, and Trudy's network may be run by Trudy: Bob must assume spoofing is possible.

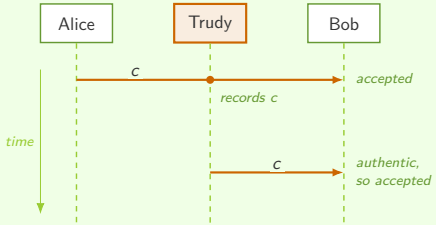
# A password, in clear and encrypted

- A password is a secret shared by Alice and whoever authenticates her. Mail, Telnet and FTP all use one.
- Sent in clear, it is read by any sniffer on the path: our FTP capture showed it word for word.
- Encrypt it with a key $K_{A\text{-}B}$ that Alice and Bob share, and Trudy no longer learns it.
- And then the key itself proves who Alice is: only she and Bob have it. The password adds little.
- The same shared secret can also key a MAC: authentication and integrity from one secret.

# The encrypted password, played back

**Trudy records the encrypted password and sends it to Bob in a new session. She cannot decrypt it. Does she get in?**

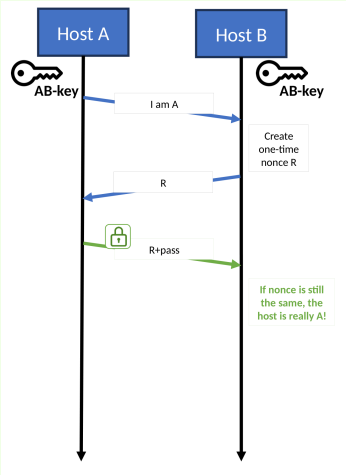
Yes. Bob decrypts it, finds the right password and accepts: Trudy used it without understanding it.




- Here $c = K_{A\text{-}B}(\text{password})$. Recording and replaying happen at different times.
- Bob cannot tell whether Alice is live: the freshness principle, broken again.
- TCP met the same problem with old SYNs: it picks an initial number, and waits to see it come back.

# A nonce proves the other side is live

- A nonce is a number a protocol uses once in a lifetime, and never again.
- Bob answers "I am A" with a fresh nonce $R$. Alice sends it back encrypted with $K_{A\text{-}B}$; Bob decrypts and compares.
- Right key: it is Alice. Right $R$: she computed it now, so she is live.
- A recording is useless tomorrow, when Bob asks for a different $R$: the password becomes a one-time password.
- The figure encrypts the password along with $R$. Harmless, but the key alone already proves who Alice is.



# The shortcut that answers its own challenge

To save messages, Alice sends her challenge with her name, and Bob answers it and sends his own in one go. What can Trudy do with two sessions open?

| # | Session | Message |
|---|---|---|
| 1 | 1 | Trudy → Bob: "I am Alice", $R_T$ |
| 2 | 1 | Bob → Trudy: $R_B$, $K_{A\text{-}B}(R_T)$ |
| 3 | 2 | Trudy → Bob: "I am Alice", $R_B$ |
| 4 | 2 | Bob → Trudy: $R_{B2}$, $K_{A\text{-}B}(R_B)$ |
| 5 | 1 | Trudy → Bob: $K_{A\text{-}B}(R_B)$ |

Bob answers his own challenge in the second session, and Trudy hands the answer back in the first: she is Alice.

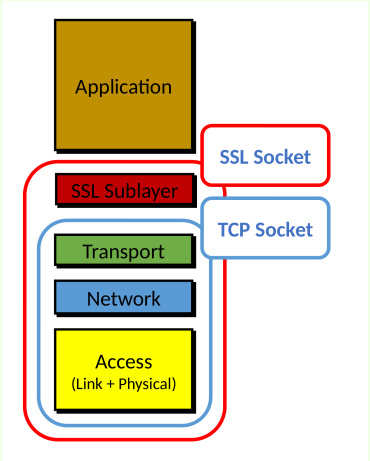
- Authentication protocols are much harder to design than they look.
- The initiator proves first; each direction has its own key; one session must never answer for another.

# From SSL to TLS

- Bob orders perfume from Alice's shop. Without protection his card number is read, his order is multiplied by ten, or the shop is Trudy's with Alice's logo.
- Put the pieces together over TCP: confidentiality, integrity, authentication of the server — and of the client, when needed.
- The first design was Netscape's SSL, the Secure Sockets Layer. Its standard successor is TLS, Transport Layer Security.
- SSL 2.0 and 3.0, TLS 1.0 and 1.1 are all retired: what runs today is TLS 1.2 and 1.3. DTLS does the same over UDP.
- Web, mail, messaging, voice: HTTPS is HTTP over TLS on port 443, the one browsers now insist on.

# TLS is a socket

- TLS secures TCP, so any application running over TCP can use it.
- Technically a sublayer of the application, above the TCP socket: the "enhancement of TCP" we met with the transport services.
- To the programmer it is a transport protocol: TCP's service, plus security.
- Its API mirrors TCP's: a TLS socket, with its own connect, read and write.
- Three phases: handshake, key derivation, data transfer.



# Your client, over TLS

The HTTP client you wrote, with `"https"` in `getaddrinfo`: the same `connect`, then TLS on top of the socket.

```
SSL_CTX *ctx = SSL_CTX_new(TLS_client_method());
SSL_CTX_set_default_verify_paths(ctx);            /* the roots of the OS  */
SSL_CTX_set_verify(ctx, SSL_VERIFY_PEER, NULL);   /* refuse a bad chain   */

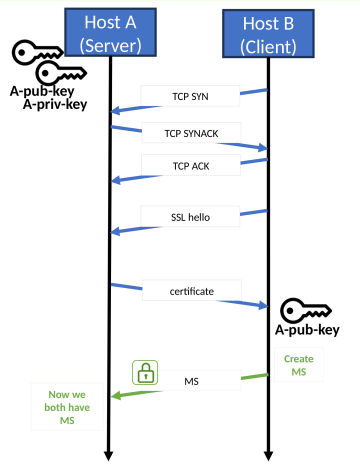
SSL *ssl = SSL_new(ctx);
SSL_set_fd(ssl, fd);                              /* your TCP socket      */
SSL_set_tlsext_host_name(ssl, "www.unina.it");    /* SNI, sent in clear   */
SSL_set1_host(ssl, "www.unina.it");               /* the name to check    */
if (SSL_connect(ssl) != 1) { ERR_print_errors_fp(stderr); exit(EXIT_FAILURE); }

SSL_write(ssl, req, strlen(req));                 /* instead of write     */
while ((n = SSL_read(ssl, buf, sizeof buf)) > 0)  /* instead of read      */
    fwrite(buf, 1, (size_t) n, stdout);
SSL_shutdown(ssl);                                /* the closure record   */
```

`SSL_connect` runs the whole handshake. Compile with `-lssl -lcrypto`; the highlighted line is the one people forget.

# The almost-TLS handshake

- Bob, the client, must open a TCP connection, check that Alice is Alice, and give her a master secret.
- After the three-way handshake Bob sends a hello; Alice answers with her certificate.
- Bob checks the CA's signature on it: the public key $K_A^+$ inside is Alice's.
- Bob picks a master secret MS, new for this session, and sends $K_A^+(\text{MS})$.
- Only Alice's private key opens it: the MS is now known to the two of them, and to nobody else.



# A valid certificate, for the wrong name

**Trudy holds a genuine certificate, signed by a real CA, for `trudy.com`. She answers when Bob connects to `www.unina.it`. What stops her?**

Only Bob comparing the name in the certificate with the name he asked for. The chain is valid; the name is not.

- A certificate proves that a key belongs to the name written in it, not that it is the server you meant.
- Browsers check and stop with a warning. Libraries check only what they are told: in OpenSSL, the highlighted line.
- Without `SSL_VERIFY_PEER` nothing is checked at all, and `curl -k` does the same on purpose.
- Each of these turns TLS back into encryption with whoever answered: the man in the middle is welcome again.

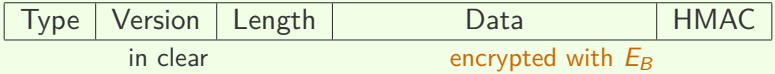
# Four keys from one secret

- The MS could key everything. It is safer to use different keys for encryption and for integrity, and for each direction.

| Session keys, derived from the MS on both sides | |
|---|---|
| $E_B$ | encryption key, data from Bob to Alice |
| $M_B$ | MAC key, data from Bob to Alice |
| $E_A$ | encryption key, data from Alice to Bob |
| $M_A$ | MAC key, data from Alice to Bob |

- $M$ is for MAC, the message authentication code: here an HMAC, a hash computed with a key.
- Separate directions: a record reflected to its sender fails its check.
- Almost-TLS slices the MS in four; real TLS uses a key derivation function.

# Records: a length, a payload, a MAC




- TCP is a byte stream: an HMAC at the end of the session would come too late. TLS cuts the stream into records.
- For each record: the HMAC with $M_B$, then data and HMAC encrypted with $E_B$, then the whole thing handed to TCP.
- The length tells the receiver where a record ends inside the TCP stream — the same header your chat protocol uses.
- The type says what the record carries: handshake, data, or the end of the session.

# Two records, swapped

**Trudy captures two segments from Bob, swaps them and fixes the TCP sequence numbers, which are in clear. What does Alice notice?**

Nothing, at first. TCP is satisfied, each record decrypts and its HMAC checks. The application gets the bytes in the wrong order.

- Each HMAC protects its own record, not the place of that record in the stream.
- The fix: a record counter from 0, fed into the HMAC together with $M_B$ and the data.
- The counter is not transmitted: both sides count. A swapped, dropped or replayed record fails its check.
- TCP's sequence number again, this time where Trudy cannot rewrite it.

# The real handshake

1. Bob sends the cipher suites he supports, and a client nonce.
2. Alice picks a symmetric cipher, a public-key algorithm and a MAC; sends her choice, her certificate, a server nonce.
3. Bob verifies the certificate and sends a pre-master secret, encrypted with Alice's public key.
4. Both compute the MS from the pre-master secret and the two nonces, then the four keys from the MS.
5. Bob sends an HMAC of all the handshake messages.
6. Alice does the same.

Steps 5 and 6 are there because step 1 travels in clear: Trudy could strike out the strong ciphers and leave a weak one. The HMAC of the whole conversation catches the downgrade.

# The same session, replayed tomorrow

**Trudy records a whole session between Bob and Alice's shop, and plays Bob's records back to Alice the next day. Every record is authentic. What happens?**

Without nonces, Alice answers exactly as before and ships the order twice. With them, her new server nonce gives new keys, and the replayed records fail their HMAC.

- Sequence numbers stop replays inside a session; nonces stop the replay of a whole session.
- The same idea that proved Alice was live now proves the session is fresh.

# A FIN that Trudy sent

**In the middle of a download, Trudy injects a TCP FIN from Bob to Alice. Does Alice know the data is incomplete?**

Only if TLS says so. A closure record must come before the FIN; a FIN without one means the end was forged.

- Releasing a TCP connection takes two FINs, and nothing authenticates them: whoever can forge a segment can end a stream.
- The closure record is a record type. Its type travels in clear, but the HMAC covers it: Trudy cannot fake it.
- In your client it is `SSL_shutdown`. Treating the end of TCP as the end of the data reopens the attack.

# What TLS 1.3 changed

- The weakness: with the secret encrypted under $K_A^+$, whoever steals Alice's private key later reads every session ever recorded.
- So TLS 1.3 drops it: the first two messages carry a Diffie-Hellman exchange, with fresh values every time.
- Diffie-Hellman alone has a man in the middle. Alice signs the handshake with her certificate's key: the authentication it lacked.
- Forward secrecy: the keys die with the session; a key stolen tomorrow decrypts nothing of today.
- One cipher encrypts and authenticates together (AES-GCM): no separate MAC keys, no CBC. The certificate itself travels encrypted.
- One round trip instead of two before the first request. QUIC carries the same handshake inside its own.

# Port 443, seen by the same sniffer

```
$ sudo tshark -i enp3s0 -f 'tcp port 443' -Y tls          same tool as for the FTP password
  4 0.0231 192.168.1.23 -> 143.225.161.30 TLSv1   583 Client Hello (SNI=www.unina.it)
  6 0.0472 143.225.161.30 -> 192.168.1.23 TLSv1.3 1514 Server Hello, Change Cipher Spec, Application Data
  8 0.0474 143.225.161.30 -> 192.168.1.23 TLSv1.3 1142 Application Data
 10 0.0519 192.168.1.23 -> 143.225.161.30 TLSv1.3  146 Change Cipher Spec, Application Data
 11 0.0521 192.168.1.23 -> 143.225.161.30 TLSv1.3  170 Application Data
 13 0.0768 143.225.161.30 -> 192.168.1.23 TLSv1.3  602 Application Data
```

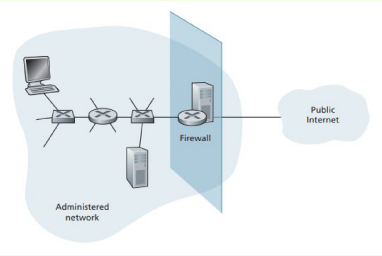
- One thing is readable: the server name, in the Client Hello, so that the server knows which certificate to send.
- After the Server Hello everything is Application Data — certificate, request and response. The `http` filter shows nothing.
- TLS 1.3 did not hide the name. An extension, ECH, encrypts it; it is only partly deployed.
- Still visible to anyone on the path: addresses, the name, sizes, timing. We will come back to that.

# Securing the DNS: two different problems

- We asked the authoritative server, and the answer came from someone else. Nothing in DNS says who wrote an answer.
- DNSSEC: each zone signs its records with its private key; an RRSIG record with the signature goes with every answer.
- The zone's public key, a DNSKEY record, is vouched for by the parent zone, up to the root: a chain, like certificates.
- A forged or poisoned answer fails the check even from a cache. Nothing is encrypted — DNS data is public — and deployment is partial.
- DoT and DoH: DNS over TLS (port 853) or over HTTPS hide the query from the path. The resolver still sees it, and signs nothing.
- DNSSEC answers who wrote the answer; encryption answers who can read the question.

# Firewalls

- Hardware and software that isolate an organization's network from the Internet, letting some packets pass and dropping the rest.
- Goals: all traffic crosses it; only what the local policy authorizes passes; the firewall itself cannot be penetrated.
- A compromised firewall gives a false sense of security, which is worse than none.
- One access point is easier to manage. Large organizations layer several, or distribute them.
- Match plus action again: the match is addresses, ports and flags; the action is to drop.



# Traditional packet filtering

- Usually on the gateway router between the organization and its ISP: everything in and out passes there.
- Each datagram is judged in isolation: pass or drop, according to rules the administrator writes.

| What a rule can look at | |
|---|---|
| IP source and destination address | TCP flags: SYN, ACK, ... |
| protocol field: TCP, UDP, ICMP, OSPF | ICMP message type |
| source and destination port | direction, and which interface |

- Nothing beyond the transport header: that is what makes it fast, and what limits it.

# From a policy to rules

| Policy | Rule |
|---|---|
| Only our Web server takes connections | drop incoming SYN unless to 130.207.244.203:80 |
| Web radios must not eat the bandwidth | drop incoming UDP, DNS excepted |
| We do not answer pings from outside | drop incoming ICMP echo requests |
| Not an amplifier for a smurf attack | drop pings to a broadcast address |
| Nobody traceroutes our network | drop outgoing ICMP TTL expired |

- Drop incoming segments with ACK $= 0$: nobody outside can open a connection, and inside clients still can.
- Rules on external addresses do not survive spoofing.
- Drop the ICMP you must, not all of it: path MTU discovery lives on "fragmentation needed".

# An access control list

| action | source address | dest address | protocol | source port | dest port | flag bit |
|---|---|---|---|---|---|---|
| allow | 222.22/16 | outside of 222.22/16 | TCP | > 1023 | 80 | any |
| allow | outside of 222.22/16 | 222.22/16 | TCP | 80 | > 1023 | ACK |
| allow | 222.22/16 | outside of 222.22/16 | UDP | > 1023 | 53 | — |

- One list per router interface, read top to bottom: the first matching rule decides, the last one denies everything else.
- Rows 1–2: out to port 80, back only with ACK set. Inside users browse; nobody connects from outside.
- Rows 3–4: DNS queries out, DNS answers in.

# An ACL for yesterday's Web

**The list is meant to let the organization browse the Web. Does a browser behind it work today?**

Hardly at all. Almost every site is on port 443, and browsers refuse plain HTTP before any packet leaves: the list blocks them all.

- And a DNS answer too large for UDP is retried over TCP port 53: dropped too, and a name fails for no visible reason.
- A rule set is a snapshot of which ports carried what, on the day it was written.
- Ports are conventions: an application may use any, and encryption hides what it carries.

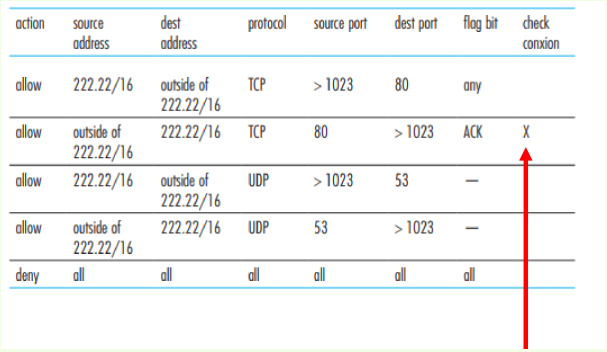
# Anything with ACK set gets in

**A packet arrives from outside: source port 80, ACK set, no connection behind it. Does row 2 let it in?**

Yes. A stateless filter judges each packet alone, and this one looks exactly like a Web reply.

- Such packets carry malformed payloads, feed a denial of service, and map the inside: whatever answers reveals a host.
- Blocking ACK segments as well stops the attack, and all Web browsing with it.
- What the filter lacks is memory: did someone inside open this connection?

# Stateful packet filtering



| action | source address | dest address | protocol | source port | dest port | flag bit | check conxion |
|---|---|---|---|---|---|---|---|
| allow | 222.22/16 | outside of 222.22/16 | TCP | > 1023 | 80 | any | |
| allow | outside of 222.22/16 | 222.22/16 | TCP | 80 | > 1023 | ACK | X |
| allow | 222.22/16 | outside of 222.22/16 | UDP | > 1023 | 53 | — | |
| allow | outside of 222.22/16 | 222.22/16 | UDP | 53 | > 1023 | — | |
| deny | all | all | all | all | all | all | |

| src | dst | sport | dport |
|---|---|---|---|
| 222.22.1.7 | 37.96.87.123 | 12699 | 80 |
| 222.22.93.2 | 199.1.205.23 | 37654 | 80 |
| 222.22.65.143 | 203.77.240.43 | 48712 | 80 |

the connection table

- The filter sees the SYN, SYNACK, ACK that open a connection, the FIN that closes it, or 60 s of silence.
- Rows marked X pass only for a connection in the table: the forged ACK finds none, the real reply does.

# The connection table on your own machine

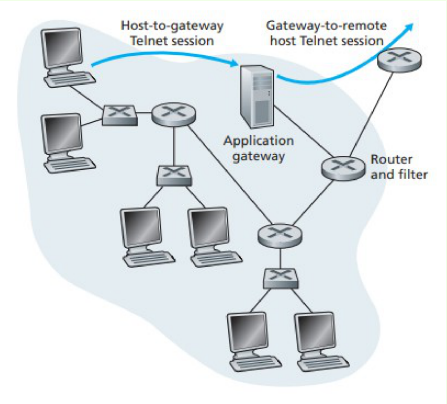
```
$ sudo conntrack -L -p tcp --dport 443                    one line, broken here
tcp  6 431996 ESTABLISHED src=192.168.1.23 dst=143.225.161.30 sport=51514 dport=443
              src=143.225.161.30 dst=192.168.1.23 sport=443 dport=51514 [ASSURED] mark=0 use=1

$ sudo nft list chain inet filter input
table inet filter {
    chain input {
        type filter hook input priority filter; policy drop;
        ct state established,related accept                the check-connection column
        iif "lo" accept
        tcp dport 22 accept
    }
}
```

- The table of the book, on a Linux machine: both directions of every connection, with its state.
- The state is the filter's opinion, built from the segments it saw — not the TCP state of either end.

# Application gateways

- "Only technicians may Telnet out" is about users: application-layer data, not in any header.
- An application gateway (ALG) is a server for one application; all its traffic must pass through it.
- The router drops every Telnet connection except those from the gateway.
- A user logs in to the gateway, which checks the permission — authorization, after authentication.
- Then it relays both ways: a server to the user, a client to the outside.



# What an application gateway costs

- One gateway per application: Telnet, HTTP and mail each need their own.
- Every byte is relayed twice: a performance penalty that grows with the number of users.
- Client software must know the gateway exists, and how to tell it where to go.
- You have already met two: an organization's mail server and its Web cache are application gateways.
- Today's Telnet gateway is an SSH bastion host: the one machine reachable from outside, where users authenticate before going further.

# Intrusion detection and prevention

- Filters read headers. Many attacks show only in the payload: deep packet inspection reads that too.
- An IDS inspects every packet and alerts the administrator. An IPS drops what it suspects.
- An IPS must sit on the path and decide at once; an IDS can watch a copy of the traffic from aside.
- Why not always block? A false alarm that kills a legitimate connection costs more than an alert someone reads.
- Network or host: a NIDS watches many machines; a HIDS watches one, after decryption and reassembly.
- It looks for port scans like `nmap`'s, DoS floods, worms, and exploits of known flaws.

# Sensors, signatures and anomalies

- Several sensors, the small boxes, each see part of the traffic and report to one processor that correlates and alarms.
- Signature-based: one rule per known attack: the most common, precise, and blind to anything new.
- Anomaly-based: a profile of normal traffic, and an alarm for what is statistically unusual: catches the new, explains little.

# A signature, read field by field

```
alert icmp $EXTERNAL_NET any -> $HOME_NET any
      (msg:"ICMP PING NMAP"; dsize: 0; itype: 8;)
```

- `alert icmp`: raise an alert on an ICMP packet, from outside the organization towards its network.
- `itype: 8` is an echo request; `dsize: 0` an empty payload. Your `ping` carries 56 bytes.
- `nmap` finds hosts on other networks with empty echo requests. On your own subnet it asks ARP instead.
- Snort's community publishes a signature within hours of a new attack; sensors download it everywhere.

# An alarm that is right 99% of the time

**The IDS spots 99% of attacks and wrongly flags 1% of normal events. One event in 100 000 is an attack. An alarm rings: how likely is it an attack?**

About one in a thousand:

$$\frac{0.00001 \cdot 0.99}{0.00001 \cdot 0.99 + 0.99999 \cdot 0.01} \approx 0.001$$

- Normal events are so many that their 1% of false alarms buries the true ones: the base-rate fallacy.
- What decides is the false positive rate, not the detection rate.
- An administrator who sees a thousand false alarms for each true one stops reading them.

# Inspecting what is encrypted

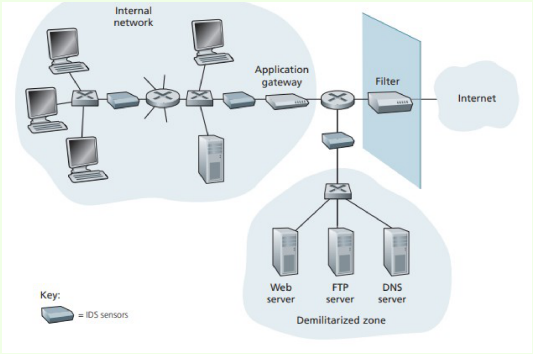
**Nearly all traffic is TLS now. What can a firewall or an IDS still read in it?**

Addresses, ports, the server name in the Client Hello, sizes and timing. The payload, where signatures match, is gone.

- One protection blinds another: encryption against inspection.
- Organizations that must inspect install their own root certificate on every machine, and a box that terminates TLS.
- That box is a man in the middle by policy: a valid certificate for any site, from a root the machines were told to trust.

# The demilitarized zone

- Servers reached from outside need looser rules than everything else.
- The DMZ is a low-security region for them: Web, mail, authoritative DNS.
- The filter guards both regions; the inside adds the gateway and stricter rules.
- A server broken in the DMZ is not yet inside: defense in depth, down to each host.
- No perimeter stops an insider.



# Summary

- A signature is a hash under a private key: anyone checks it, one could have made it. A certificate is a CA's signature.
- Authentication needs a secret and freshness: a nonce shows the other side is there now.
- TLS: a certificate to trust the server, a handshake for keys, records with a MAC and a counter. Version 1.3 signs Diffie-Hellman.
- Firewalls: filter headers, alone or with a connection table; application gateways filter users and content.
- IDS: read payloads and alert. Their enemy is the false positive, and encryption blinds them.
- TLS hides what you say, not whom you talk to. Next: hiding that too.